

Assignment 4: Multi-Client Chat Server using TCP

NAME : PIYUSH KUMAR

ROLL : CSB24051

1. Objective

Develop a multi-client chat application using TCP sockets where multiple clients can communicate through a centralized server. The assignment aims to introduce concurrent server design, client management, message broadcasting, and performance evaluation.

2. Network Topology

The experiment was performed using **Mininet**.

Topology Used

Host	Function
------	----------

h1	Chat Server
----	----------------

h2	Client A
----	----------

h3	Client B
----	----------

h4	Client C
----	----------

3. Program Design

Server

The server performs the following tasks:

- Creates a TCP socket.
- Binds to Port **5000**.
- Waits for incoming client requests.
- Accepts multiple client connections.
- Creates one thread for each client.
- Registers usernames.
- Broadcasts received messages to every connected client.
- Maintains active client list.
- Logs connection and disconnection events.
- Stores all chat messages in **chat_log.txt**.

Client

The client performs the following tasks:

- Creates TCP socket.
- Connects to server.
- Sends username.
- Sends chat messages.
- Receives broadcast messages.
- Displays received messages continuously.

4. Experimental results

The experiment was performed for:

- 1 Client
- 2 Clients
- 3 Clients

Sample Results

Client s	Total Messages	Avg Delivery Time (ms)	Throughput (msg/sec)
1	20	0.004	0.161
2	40	0.027	0.239
3	60	0.046	0.274

5. Wireshark Verification

Filter Used : **tcp.port == 5000**

a. Three way handshake

tcp.port == 5000						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=83 Len=8 TSval=3291584655 TSecr=2219264360
2	0.000137875	10.0.0.1	10.0.0.2	TCP	69	5000 → 56986 [PSH, ACK] Seq=1 Ack=9 Win=85 Len=3 TSval=2219285682 TSecr=3291584655
3	0.000419795	10.0.0.2	10.0.0.1	TCP	66	56986 → 5000 [ACK] Seq=9 Ack=4 Win=83 Len=0 TSval=3291584656 TSecr=2219285682
4	2.115132074	10.0.0.2	10.0.0.1	TCP	74	56986 → 5000 [PSH, ACK] Seq=9 Ack=4 Win=83 Len=8 TSval=3291586771 TSecr=2219285682

b. Chat Message Packet

5	2.115267360	10.0.0.1	10.0.0.2	TCP	69 5000 → 56986 [PSH, ACK] Seq=4 Ack=17 Win=85 Len=3 TSval=2219287797 TSecr=2219287797
6	2.115315927	10.0.0.2	10.0.0.1	TCP	66 56986 → 5000 [ACK] Seq=17 Ack=7 Win=83 Len=0 TSval=3291586771 TSecr=2219287797
7	4.451898702	10.0.0.2	10.0.0.1	TCP	74 56986 → 5000 [PSH, ACK] Seq=17 Ack=7 Win=83 Len=8 TSval=3291589107 TSecr=2219287797
8	4.452056608	10.0.0.1	10.0.0.2	TCP	69 5000 → 56986 [PSH, ACK] Seq=7 Ack=25 Win=85 Len=3 TSval=2219290133 TSecr=3291589107
9	4.452196743	10.0.0.2	10.0.0.1	TCP	66 56986 → 5000 [ACK] Seq=25 Ack=10 Win=83 Len=0 TSval=3291589107 TSecr=2219290133
10	6.840839699	10.0.0.2	10.0.0.1	TCP	74 56986 → 5000 [PSH, ACK] Seq=25 Ack=10 Win=83 Len=8 TSval=3291591496 TSecr=2219290133
11	6.840999066	10.0.0.1	10.0.0.2	TCP	69 5000 → 56986 [PSH, ACK] Seq=10 Ack=33 Win=85 Len=3 TSval=2219292522 TSecr=3291591496
12	6.841046101	10.0.0.2	10.0.0.1	TCP	66 56986 → 5000 [ACK] Seq=33 Ack=13 Win=83 Len=0 TSval=3291591496 TSecr=2219292522
13	9.121022373	10.0.0.2	10.0.0.1	TCP	74 56986 → 5000 [PSH, ACK] Seq=33 Ack=13 Win=83 Len=8 TSval=3291593776 TSecr=2219292522
14	9.121120694	10.0.0.1	10.0.0.2	TCP	69 5000 → 56986 [PSH, ACK] Seq=13 Ack=41 Win=85 Len=3 TSval=2219294803 TSecr=3291593776
15	9.121156214	10.0.0.2	10.0.0.1	TCP	66 56986 → 5000 [ACK] Seq=41 Ack=16 Win=83 Len=0 TSval=3291593777 TSecr=2219294803
16	11.170953390	10.0.0.2	10.0.0.1	TCP	74 56986 → 5000 [PSH, ACK] Seq=41 Ack=16 Win=83 Len=8 TSval=3291595826 TSecr=2219294803
17	11.171085997	10.0.0.1	10.0.0.2	TCP	69 5000 → 56986 [PSH, ACK] Seq=16 Ack=49 Win=85 Len=3 TSval=2219296852 TSecr=3291595826
18	11.171136257	10.0.0.2	10.0.0.1	TCP	66 56986 → 5000 [ACK] Seq=49 Ack=19 Win=83 Len=0 TSval=3291595826 TSecr=2219296852

c. Broadcast Packet

584	963.017669778	10.0.0.1	10.0.0.3	TCP	66 5000 → 35196 [FIN, ACK] Seq=58 Ack=153 Win=43520 Len=0 TSval=4217660326 TSecr=3670778517
585	963.017741221	10.0.0.1	10.0.0.4	TCP	66 5000 → 46416 [FIN, ACK] Seq=396 Ack=159 Win=43520 Len=0 TSval=231850651 TSecr=2753056240
586	963.017752065	10.0.0.1	10.0.0.2	TCP	66 5000 → 52902 [FIN, ACK] Seq=747 Ack=159 Win=43520 Len=0 TSval=2220215398 TSecr=3292484673

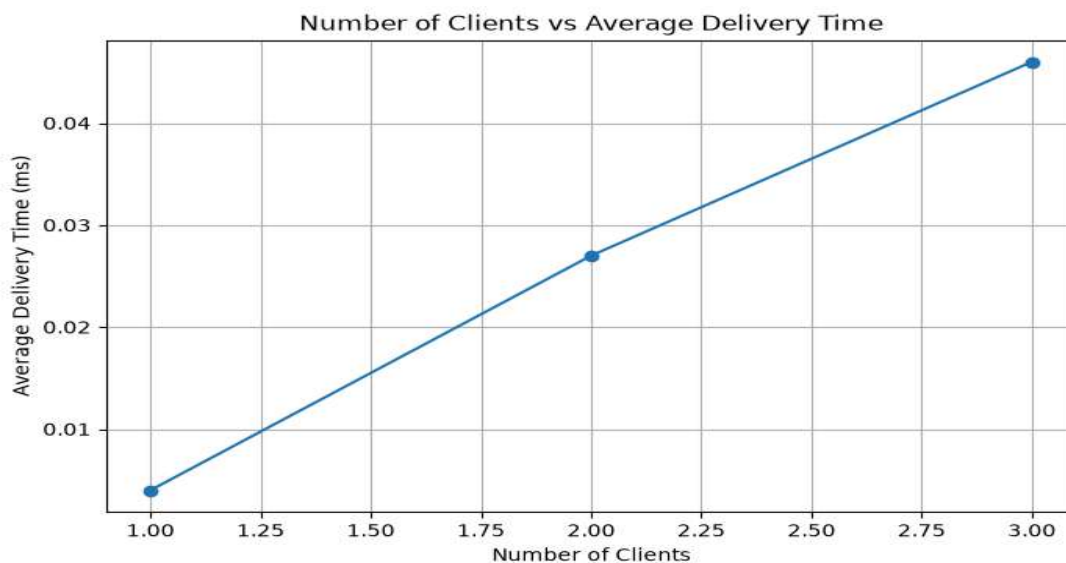
d. TCP Connection Termination

587	963.065877325	10.0.0.2	10.0.0.1	TCP	66 52902 → 5000 [ACK] Seq=159 Ack=397 Win=42496 Len=0 TSval=3292514429 TSecr=2220215398
588	963.065943058	10.0.0.4	10.0.0.1	TCP	66 46416 → 5000 [ACK] Seq=159 Ack=397 Win=42496 Len=0 TSval=2753085987 TSecr=231850651
589	963.065962547	10.0.0.3	10.0.0.1	TCP	66 35196 → 5000 [ACK] Seq=153 Ack=59 Win=42496 Len=0 TSval=3670808265 TSecr=4217660326

6. Graph Analysis

Graph 1

Number of Clients vs Average Delivery Time

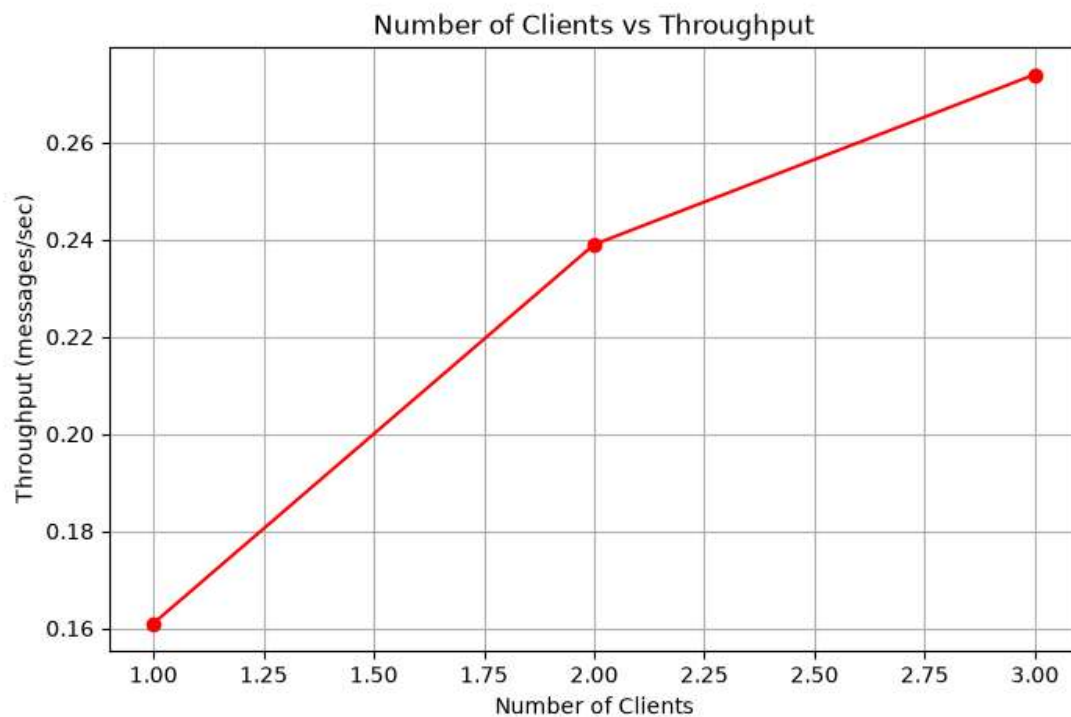


Observation

- Delivery time increased gradually with more clients.
- Thread scheduling and message broadcasting contributed to the delay.

Graph 2

Number of Clients vs Throughput



Observation

- Throughput improved with more active clients.
- Multi-threading efficiently handled concurrent communication.

7. Reflection Answers

Q1. Why is a multi-threaded server preferred over a single-threaded server?

Answer

A multi-threaded server can handle multiple clients simultaneously. Each client is assigned a separate thread, so one slow client does not block others. This improves responsiveness, resource utilization, and scalability.

Q2. What challenges arise when multiple clients communicate simultaneously?

Answer

Simultaneous communication may cause synchronization issues, race conditions, thread management overhead, increased memory usage, and message ordering problems. Proper synchronization mechanisms are required to avoid data inconsistency.

Q3. How does TCP ensure reliable delivery of chat messages?

Answer

TCP provides reliable communication using sequence numbers, acknowledgements (ACKs), retransmission of lost packets, flow control, error detection with checksums, and ordered packet delivery. These mechanisms ensure that chat messages reach the destination accurately and in sequence.

Q4. What information from Wireshark helped verify the application behavior?

Answer

Wireshark verified:

- TCP three-way handshake,
- successful message transmission,
- server broadcast packets,
- TCP connection termination,
- source and destination IP addresses,
- source and destination port numbers,
- TCP sequence and acknowledgement numbers.

8. Conclusion

The experiment successfully demonstrated the implementation of a **multi-client TCP chat server** using Python sockets and multi-threading. Multiple clients communicated concurrently through a centralized server, and all messages were broadcast correctly. Connection events and chat messages were logged successfully. Performance evaluation showed that throughput increased with more clients, while message delivery time also increased due to concurrent processing. Wireshark packet captures confirmed correct TCP connection establishment, message transmission, broadcasting, and graceful connection termination. Overall, the experiment demonstrated the reliability and efficiency of TCP-based client-server communication in a multi-threaded environment.